



雨
龍

Інформаційна система «Читальний зал»

Виконала: Ямащіта Р.Ю. 6.1213-2пi

Керівник: доцент кафедри програмної інженерії, к.ф.-м.н., Мильцев О.М.

Мета та постановка задачі

*Мета роботи – аналіз предметної області,
проєктування та розробка вебзастосунку для
читання азійських романів*

Задачі, які необхідно розв'язати для досягнення поставленої мети:

- проаналізувати предметну область «Азійські романи»;
- проаналізувати обрані засоби розробки;
- сформулювати вимоги до вебзастосунку;
- реалізувати базу даних засобами PostgreSQL;
- спроектувати вебзастосунок згідно вимог;
- розробити вебзастосунок за допомогою обраних засобів;
- протестувати вебзастосунок.



Опис предметної області

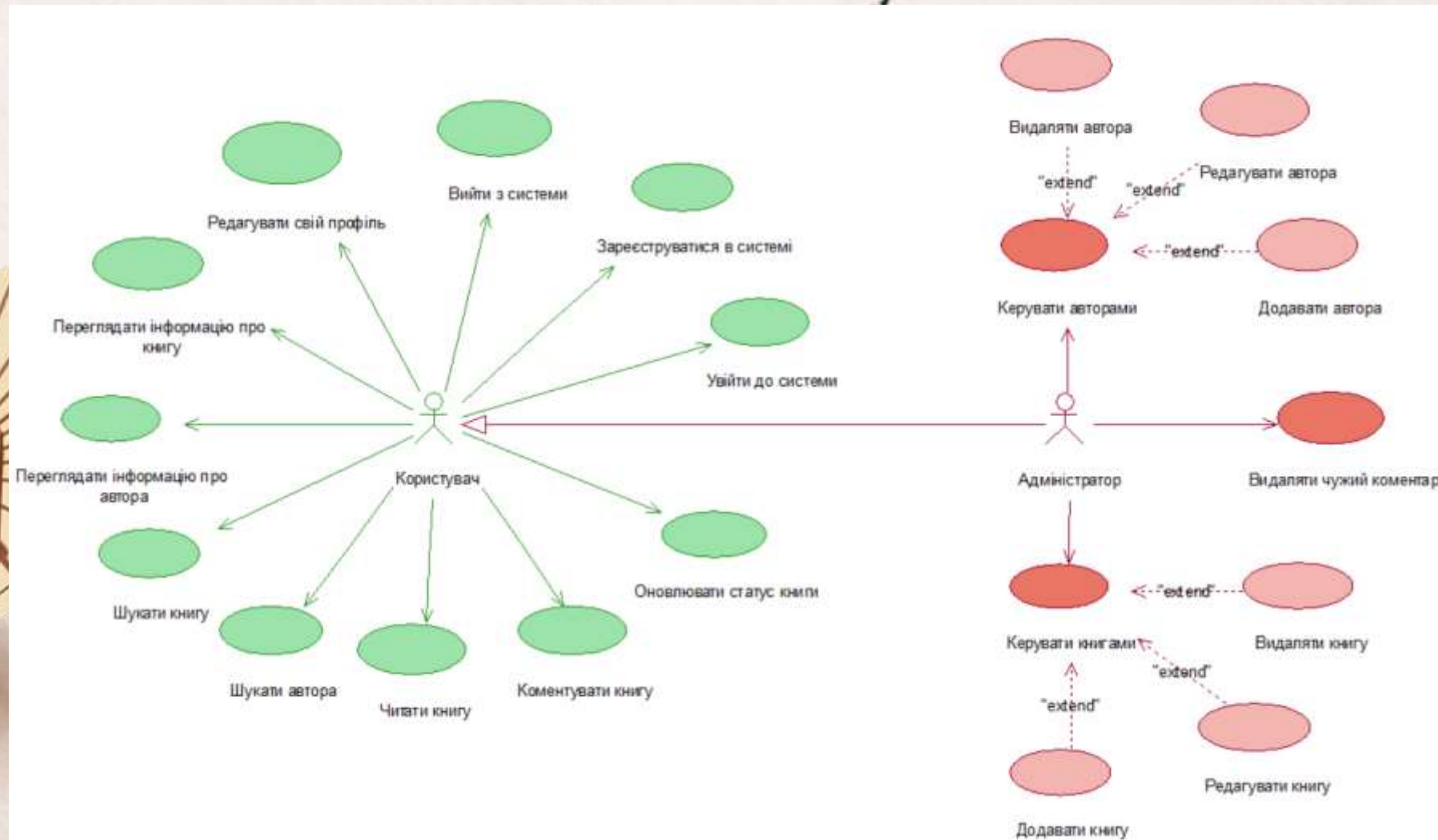
*Предметною областю є азійські романи,
які мають неповторний стиль, що
приваблює аудиторію різноманітної
статі та віку*

Користувач має мати можливість реєструватися,
увійти до системи та переглядати інформацію.

Адміністратор має всі ті ж самі функції з
можливістю додавати, оновлювати та видаляти
інформацію



Діаграма прецедентів



Засоби розробки



Були також використані бібліотеки: *ePub.js*, *multer*, *JWT Decode*, *FontAwesome*, *PostgreSQL Client*, тощо

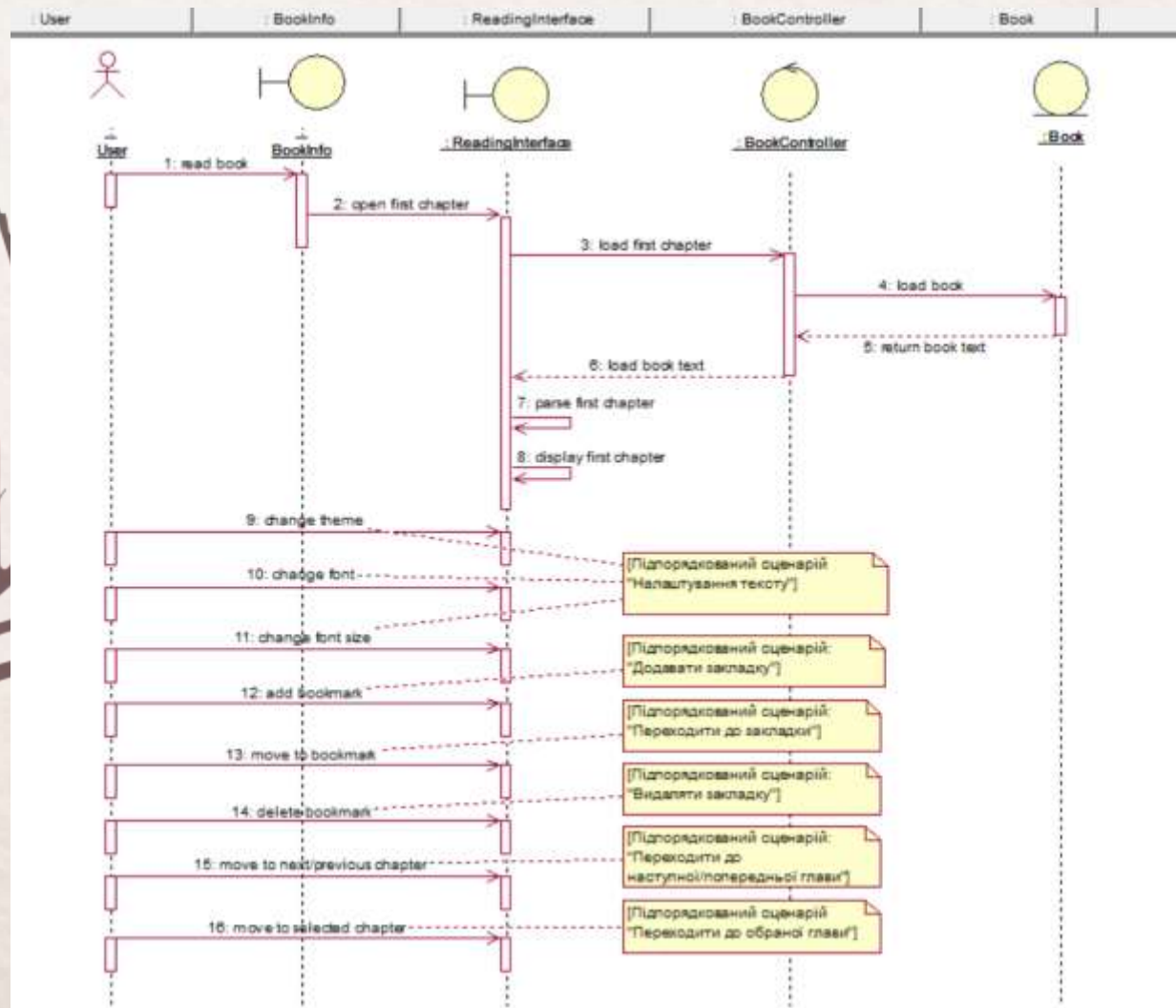
Опис архітектури

*Використано архітектурний патерн MVC
(Model–View–Controller), який забезпечує
розділення відповідальностей між
компонентами системи*

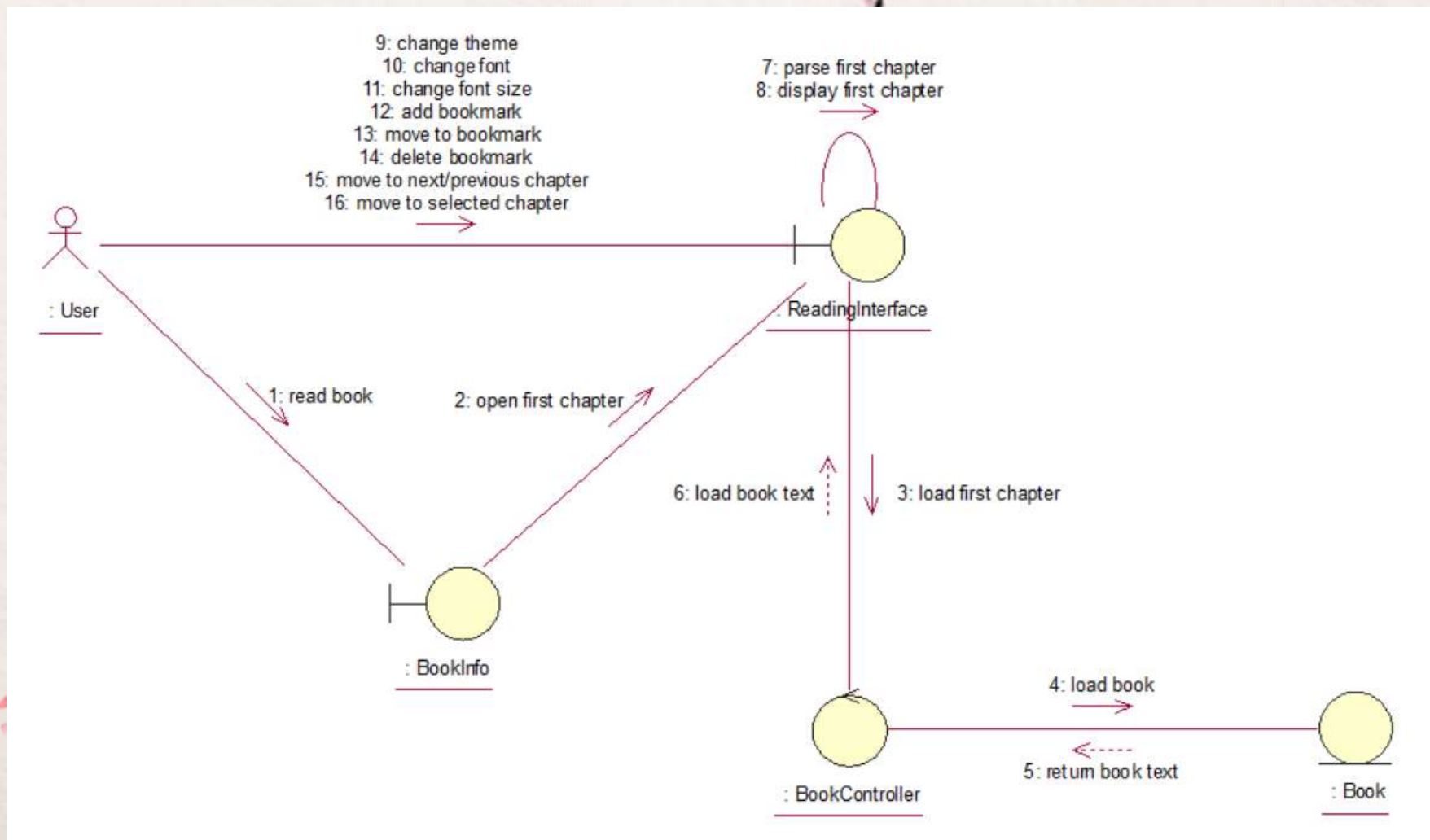
Модель представлена класами сутностей, що описують структуру даних, контролери з сервісами реалізують обробку запитів і повернення результатів, а роль представлення виконує клієнтська частина на базі React



Діаграма послідовностей



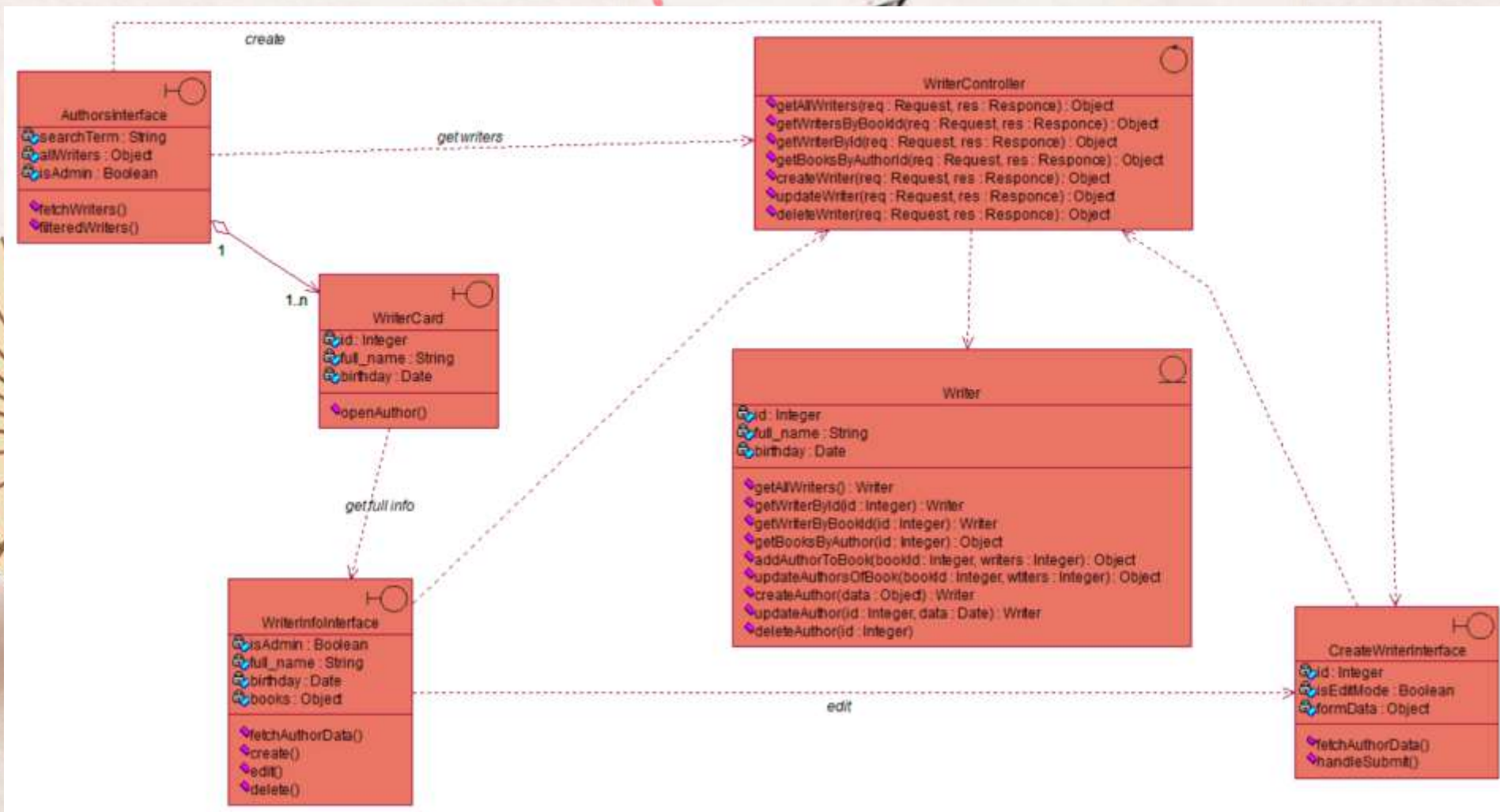
Діаграма кооперації



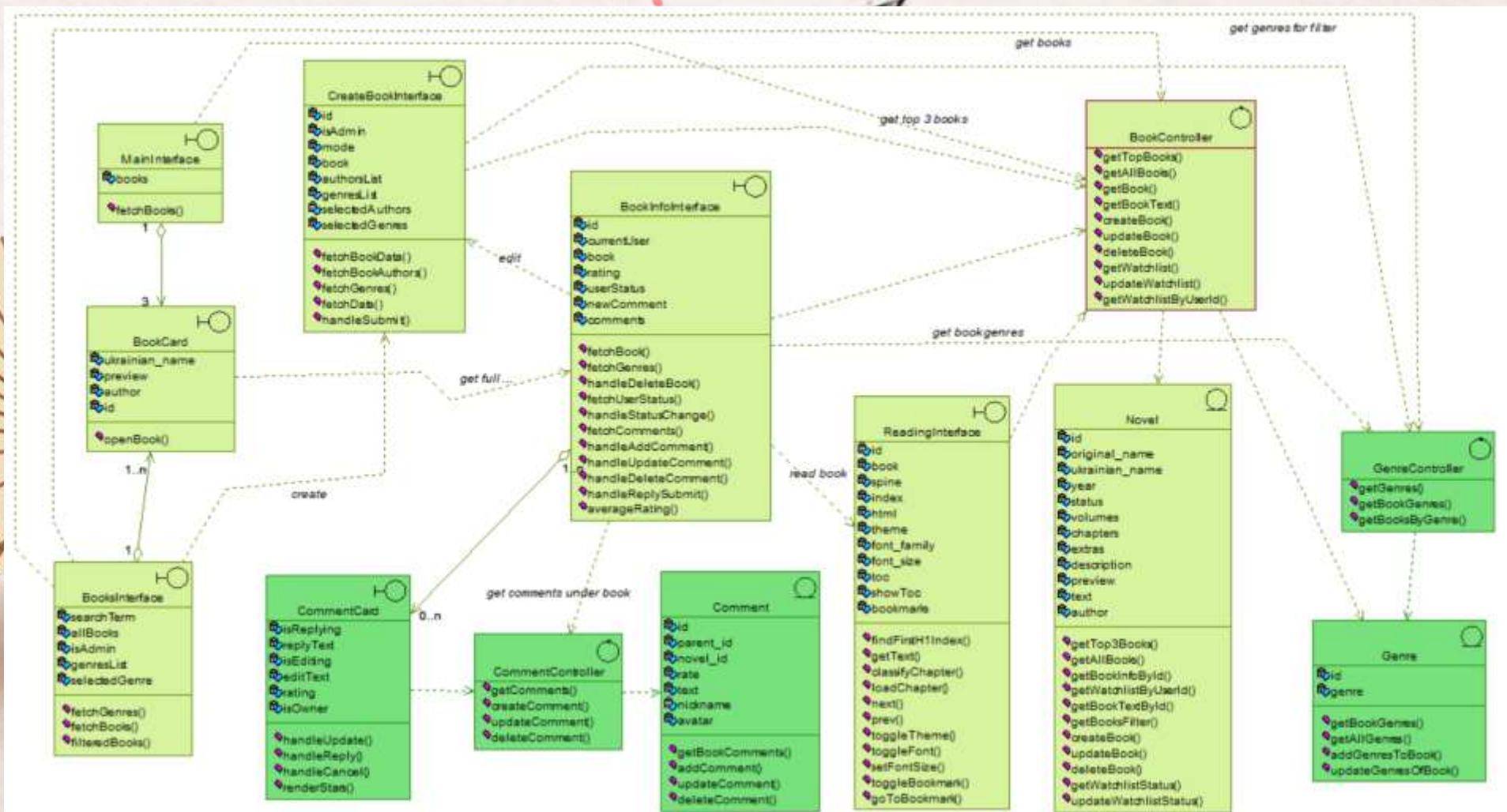
ама класів



Діаграма класів



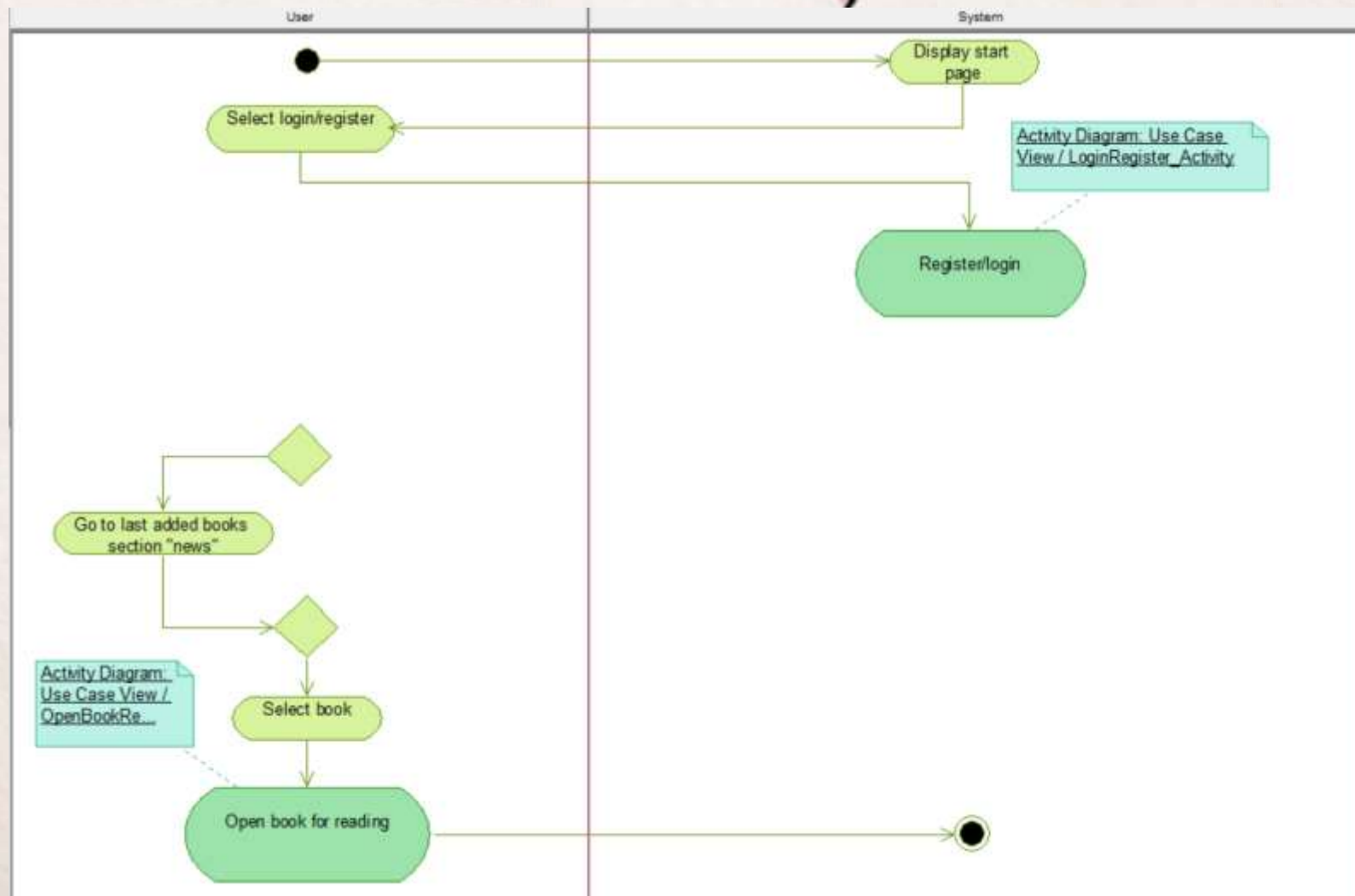
Діаграма класів



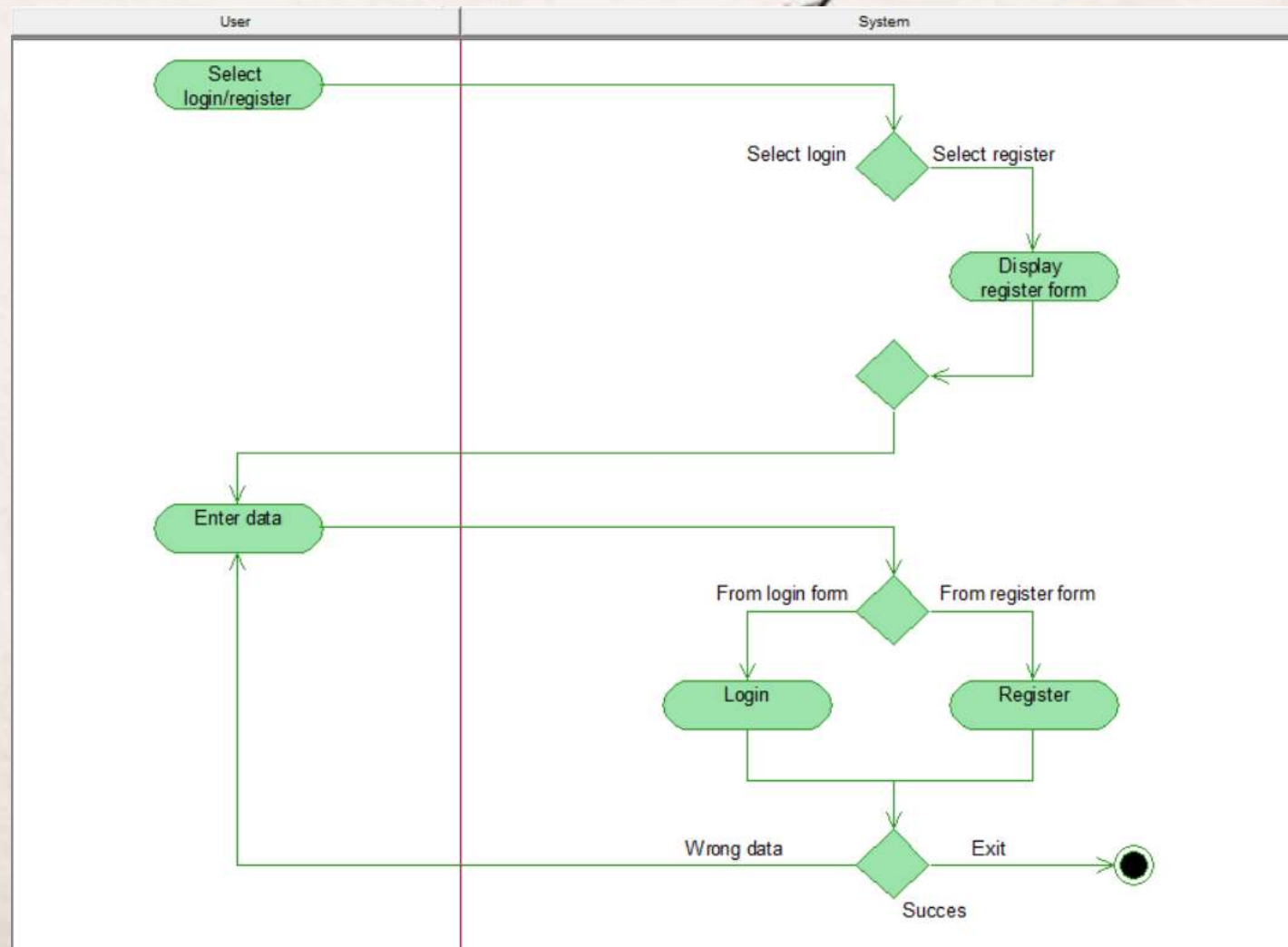
Діаграма класів



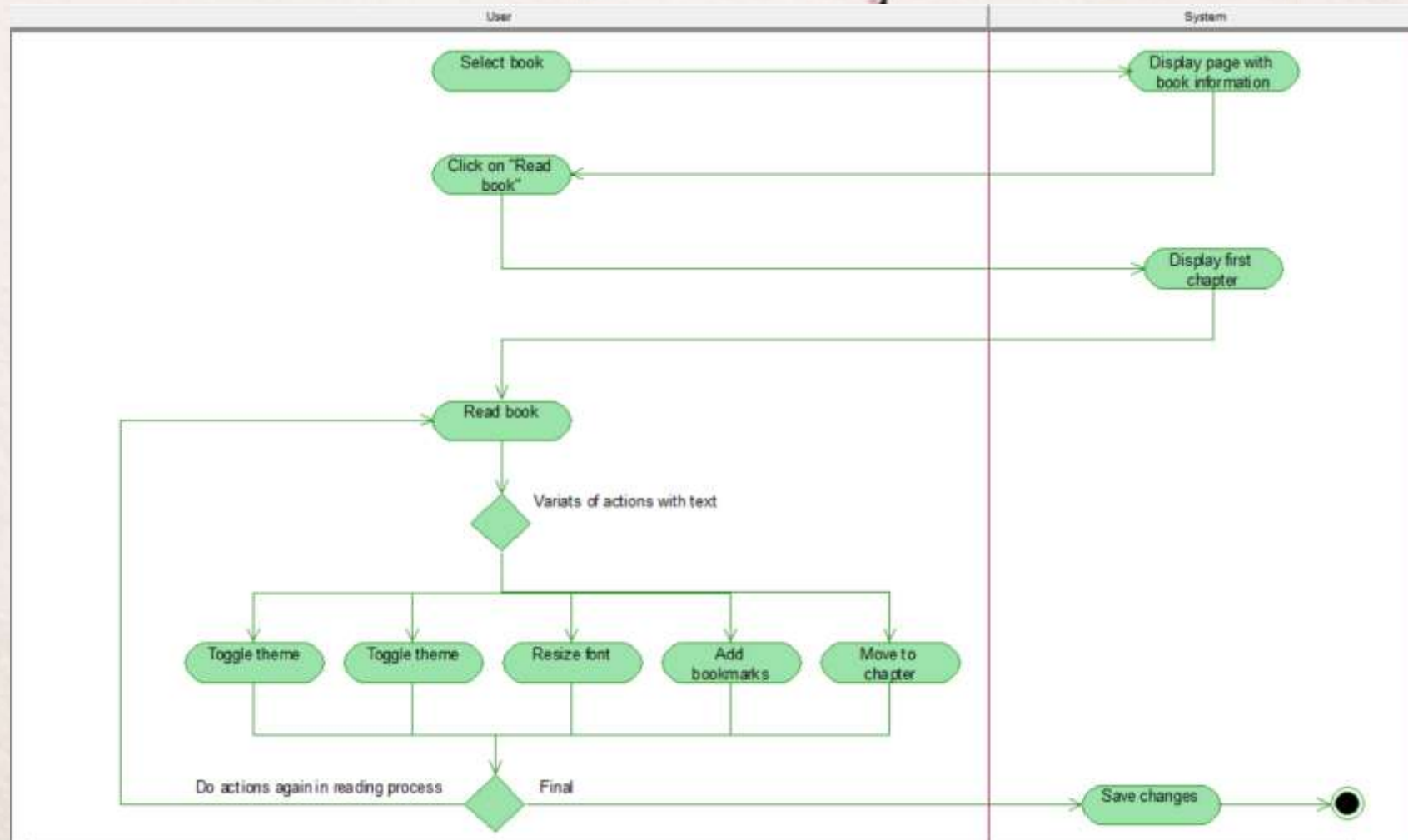
Діаграма діяльності



Діаграма діяльності



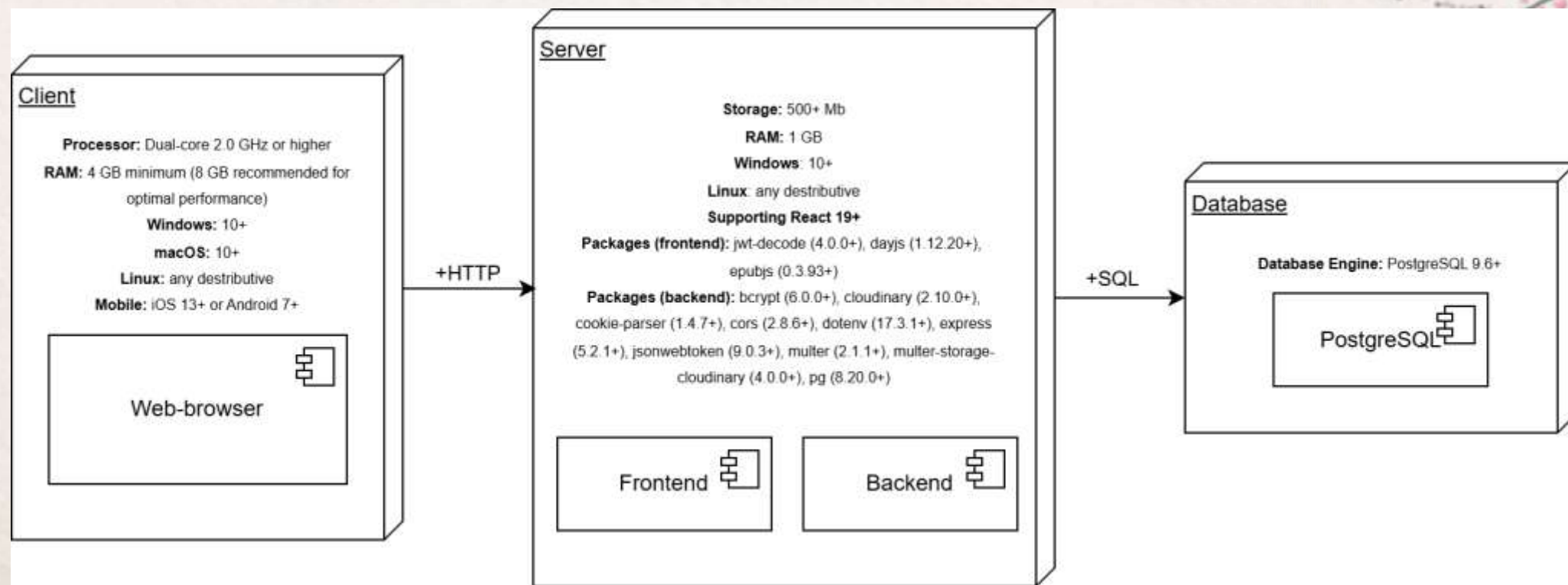
Діаграма діяльності



A minimalist illustration of a bare, brown tree branch with a few small red blossoms, set against a light beige background. The branch is dark brown and extends from the bottom left towards the top right. Several thin, dark brown lines represent smaller branches or twigs. A few small, red, teardrop-shaped blossoms are scattered along the main branch and the smaller twigs. The background is a light beige color with a subtle, textured pattern.



Діаграма розгортання





бази даних



Автентифікація

```
6 const requestWithRefresh = async (url, options = {}) => {
7   // Функція для виконання запиту з автоматичним оновленням токена при 401 помилці
8   const makeRequest = (token) => {
9     fetch(url, {
10       ...options,
11       headers: {
12         ...(options.headers || {}),
13         Authorization: `Bearer ${token || localStorage.getItem("token")}`,
14       },
15     });
16   };
17   // Виконуємо запит вперше з поточним токеном
18   let res = await makeRequest();
19
20   if (res.status === 401) {
21     if (!isRefreshing) {
22       isRefreshing = true;
23       try {
24         const refreshRes = await fetch("http://localhost:5000/api/refresh", {
25           method: "POST",
26           credentials: "include",
27         });
28
29         if (refreshRes.ok) {
30           const data = await refreshRes.json();
31           localStorage.setItem("token", data.accessToken);
32           isRefreshing = false;
33           // У кожний елемент масиву пробрасую новий токен, щоб вони могли повторити свої запити з оновленим токеном по черзі
34           onRefreshed(data.accessToken);
35         } else {
36           throw new Error("Refresh failed");
37         }
38       } catch (err) {
39         isRefreshing = false;
40         localStorage.removeItem("token");
41         window.location.href = "/";
42         return res;
43       }
44     }
45   }
46
47   // Створюємо проміс, який буде чекати, поки токен оновиться, і потім повторить оригінальний запит з новим токеном
48   const retryOriginalRequest = new Promise((resolve) => {
49     // resolve як купер, що чекає. Залишаємо йому функцію subscribeTokenRefresh, яка буде викликана, коли токен оновиться. Вона отримає новий токен і виконає makeRequest
50     subscribeTokenRefresh((token) => {
51       resolve(makeRequest(token));
52     });
53   });
54   return retryOriginalRequest;
55 }
```

Автентифікація

```
3 export const authMiddleware = (req, res, next) => {
4   const authHeader = req.headers.authorization;
5   const token = authHeader?.split(" ")[1];
6
7   if (!token) return res.status(401).json({ message: "Токен відсутній" });
8
9   try {
10    const decoded = jwt.verify(token, process.env.JWT_SECRET);
11    req.user = decoded;
12    next();
13  } catch (err) {
14    // Якщо токен прострочений або невалідний — просто 401
15    // Клієнт отримає цю помилку і зрозуміє, що треба йти на /refresh
16    return res.status(401).json({ message: "Token expired or invalid" });
17  }
18 };
```

```
46 export const refresh = async (req, res) => {
47   try {
48     const refreshToken = req.cookies?.refreshToken;
49     if (!refreshToken) return res.status(401).json({ message: "No refresh token" });
50
51     const decoded = jwt.verify(refreshToken, process.env.REFRESH_SECRET);
52
53     // Об'єднуємо userId та admin в один об'єкт payload
54     const newAccessToken = jwt.sign(
55       { userId: decoded.userId, admin: decoded.admin },
56       process.env.JWT_SECRET,
57       { expiresIn: "1h" }
58     );
59
60     res.json({ accessToken: newAccessToken });
61   } catch (err) {
62     res.status(403).json({ message: "Refresh token invalid" });
63   }
64 };
```

Висновки



1

Було повністю спроектовано та реалізовано вебзастосунок

2

Вебзастосунок повністю відповідає поставленим вимогам

3

Вдосконалено практичні навички проектування та розроблення сучасних вебзастосунків



雨
龍

Дякую за увагу!

Для презентації використано шаблон Freerppt7.com

